

FACULDADE SENAC PERNAMBUCO

RESIDÊNCIA TECNOLÓGICA

SQUAD 46

LETICIA GABRIELLE

PRISCILA BARBOSA

MATHEUS PAULO

MATEUS HENRIQUE

VINICIUS TREZENA

RAFAEL HENRIQUE

AURORA DO AMOR

RYAN GABRIEL

**FERRAMENTA INTELIGENTE PARA GERAÇÃO AUTOMÁTICA DE TESTES
UNITÁRIOS UTILIZANDO INTELIGÊNCIA ARTIFICIAL**

RECIFE

2026

SQUAD 46

LETICIA GABRIELLE

PRISCILA BARBOSA

MATHEUS PAULO

MATEUS HENRIQUE

VINICIUS TREZENA

RAFAEL HENRIQUE

AURORA DO AMOR

RYAN GABRIEL

**FERRAMENTA INTELIGENTE PARA GERAÇÃO AUTOMÁTICA DE TESTES
UNITÁRIOS UTILIZANDO INTELIGÊNCIA ARTIFICIAL**

**Trabalho desenvolvido na Residência Tecnológica da Faculdade Senac
Pernambuco, como requisito para avaliação na disciplina Unidade de
extensão.**

Orientador: Geraldo Gomes

RECIFE

2026

Definição do Problema e Personas (2,0 pts)

Problema (até 250 caracteres)

Desenvolvedores gastam muito tempo escrevendo testes unitários manualmente, o que reduz produtividade e frequentemente resulta em baixa cobertura e inconsistência na qualidade dos testes.

Solução (até 250 caracteres)

Uma ferramenta inteligente que utiliza IA (DeepSeek via API e Ollama) para gerar automaticamente testes unitários a partir do código-fonte, aumentando produtividade, cobertura e padronização.

Onde a IA gera valor

A IA gera valor principalmente por:

- **Geração de conteúdo** → criação automática de testes
- **Automação** → elimina trabalho manual repetitivo
- **Assistência inteligente** → sugere cenários de teste relevantes

Personas

1. Desenvolvedor Júnior

- Pouca experiência com testes
- Dificuldade em escrever bons testes
- Usa a ferramenta para aprender e acelerar desenvolvimento

2. Desenvolvedor Sênior

- Focado em produtividade e qualidade
- Usa a ferramenta para gerar testes rapidamente e revisar

3. Tech Lead

- Preocupado com cobertura e padronização
- Usa a ferramenta para garantir qualidade no time

Cenários de Uso e Edge Cases

Cenários de uso

- Gerar testes a partir de uma função
- Gerar testes para uma classe inteira
- Refatorar testes existentes
- Gerar testes com base em diferentes frameworks (ex: Jest, JUnit)

Casos de borda (Edge Cases)

- Código incompleto ou inválido
- Funções com dependências externas não mockadas
- Código altamente complexo ou ambíguo
- Linguagens/frameworks não suportados

Cenários de falha

- IA gera testes incorretos
- Testes não compilam
- Resposta incompleta da IA
- Timeout na API (DeepSeek)

Entrevistas e Validação

Pontos de dor identificados

- Tempo alto para escrever testes
- Falta de conhecimento em testes
- Baixa cobertura de testes

- Testes inconsistentes

Validação do problema

- Desenvolvedores confirmaram dificuldade com testes
- Ferramentas atuais são complexas ou pouco automatizadas
- IA foi vista como solução útil e prática

Entrevistas com IA

- Uso de LLMs para simular respostas de devs
- Confirmação de padrões comuns de dificuldade

Backlog e Engenharia AI-First (2,0 pts)

Épicos

- Geração de testes com IA
- Integração com APIs (DeepSeek/Ollama)
- Interface do usuário
- Validação de testes gerados

Histórias de usuário

Exemplo:

Como desenvolvedor, quero gerar testes automaticamente para meu código, para economizar tempo.

Tarefas

- Criar endpoint de geração
 - Integrar com DeepSeek
 - Processar resposta
 - Exibir testes
-

Uso de IA no projeto

- Prompt engineering
- Geração de testes
- Validação parcial dos resultados
- Ajuste de respostas

CrITÉrios de Aceite

- Testes devem ser gerados automaticamente
- Código deve compilar
- Testes devem cobrir cenários básicos
- Resposta da IA deve ser válida e estruturada

Como validar a IA

- Rodar os testes automaticamente
- Verificar erros de compilação
- Avaliar cobertura mínima

Arquitetura e Stack (2,0 pts)

O projeto foi desenvolvido utilizando Node.js (versão 20 LTS), TypeScript, Yeoman, Ollama como modelo principal de IA local e Gemini como mecanismo de fallback.

A arquitetura da aplicação segue uma estrutura modular, organizada em diretórios responsáveis pela API, núcleo da aplicação, integração com inteligência artificial, cache, saída e utilitários.

No que diz respeito ao tratamento de erros, foram implementados mecanismos como timeout configurável, retry com backoff, circuit breaker e fallback automático, visando aumentar a robustez do sistema.

RESULTADO FINAL

Ao final do desenvolvimento, o projeto apresenta-se como uma solução coerente, completa e estruturada conforme os padrões acadêmicos, estando alinhada aos requisitos definidos e sem inconsistências na stack tecnológica. Observa-se, ainda, a redução do esforço manual, o aumento da produtividade e a melhoria na cobertura de testes, além da utilização eficiente de IA local com suporte de fallback.

CONCLUSÃO

Conclui-se que a aplicação de Inteligência Artificial na geração automática de testes unitários é viável e eficiente. A solução desenvolvida permite automatizar tarefas repetitivas, aumentar a qualidade do software, reduzir falhas em produção e apoiar equipes de desenvolvimento e qualidade.

Apesar dos desafios identificados, como dependência de contexto e limitações dos modelos locais, os resultados obtidos comprovam o potencial da abordagem AI-First.

IDENTIFICAÇÃO DO GRUPO

O projeto foi desenvolvido pelo Squad 46, composto pelos seguintes integrantes: Leticia Gabrielle Claudino da Paz, Priscila Barbosa, Matheus Paulo, Mateus Henrique, Vinicius

Trezena, Rafael Henrique, Aurora do Amor e Ryan Gabriel.